Assignment Kit for Coding Standard

Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview

This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites
- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to
- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements	Produce, document, and submit a completed coding standard that calls for quality coding practices. For LOC counting, ensure that a separate physical source line is used for each logical line of code. Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example
- Since it is an example, tailor it to meet your personal needs.
- If you have an existing organizational standard, consider using it for the PSP exercises.

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	/ **/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ / **/
Listing Contents	Provide a summary of the listing contents
Contents Example	/ **/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ / **/

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	/ ** */ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ / ** */
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	If(record_count > limit) /* have all records been processed? */
Bad Comment	If(record_count > limit) /* check if record count exceeds limit */
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	/ ** */ /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ / ** */
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.

Indenting Example	while (miss_distance > threshold)
	{
	  success_code = move_robot (target _location);
	  if (success_code == MOVE_FAILED)
	  {
	    printf("The robot move has failed.\n");
	  }
	}

Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization	#define DEFAULT-NUMBER-OF-STUDENTS 15

Examples	int class-size = DEFAULT-NUMBER-OF-STUDENTS;

Evaluation criteria and suggestions

Evaluation criteria	Your standard must be • complete • legible
Suggestions	Keep your standards simple and short. Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

<table>
<tr><td>Purpose</td><td>El propósito de este estándar es:
Establecer un conjunto consistente de prácticas de codificación para el desarrollo del Programa 4A (PSP2), que implementa la integración numérica mediante la regla de Simpson aplicada a la distribución t de Student.
Facilitar el conteo de líneas de código (LOC) siguiendo el estándar PSP.
Garantizar claridad, legibilidad, consistencia y reusabilidad del código fuente.
Este programa se creó para poder calcular la integral de la distribución t porque no existe una fórmula rápida para hacerlo. La regla de Simpson nos ayuda a obtener ese valor de forma aproximada y precisa. Este valor se usa en el PSP para poder hacer predicciones y análisis estadísticos.</td></tr>
<tr><td>Program Headers</td><td>Begin all programs with a descriptive</td></tr>
<tr><td>Header Format</td><td><pre>
 /*
 * <NombreDelArchivo>.java
 * Autor: Marely Jacome
 * Fecha: 2025-12-02
 * Versión: 2.0
 * Descripción: Descripción breve del propósito del
archivo.
 */
</pre></td></tr>
<tr><td>Listing Contents</td><td>Provide a summary of the listing contents.</td></tr>
</table>

Contents Example	``` /* * Listing Contents: * - Reuse Instructions * - Atributos de la clase * - Métodos públicos * - Métodos privados * - Dependencias con otras clases */ ```
Reuse Instructions	• Describe how the program is used. Provide the declaration format, parameter values and types, and parameter limits. • Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.

<table>
<tr><td>Reuse Example</td><td>

```
/*
 * Reuse Instructions:
 *   - Usar computeIntGamma(n) para valores enteros.
 *   - Usar computeDblGamma(x) para valores .5 (half-
integers).
 *   - Gamma(0.5) = sqrt(pi)
 *   - Gamma(n) = (n-1)!
 *   - Gamma(x) = (x-1)*Gamma(x-1)
 */
```

Para SimpsonIntegration.java

- Implementa todo el procedimiento del UML:
  - computeW
  - computeXi
  - computeFirstBaseTerms
  - computeExponent
  - computeCoefficient
  - computeFxi
  - computeFinalTerms
  - computeFinalValue
- Recibe: (num_seg, x, dof)
- Devuelve: $p = P(T \leq x)$

</td></tr>
<tr><td>Identifiers</td><td>Use descriptive names for all variables, function names, constants, and other identifiers. Avoid abbreviations or single letter variables.</td></tr>
</table>

Identifier Example	Variables permitidas: • dblX • dblW • dblE • intNumSeg • dblTotXi[] • dblFinalTerms[] Variables NO permitidas: • x • y • a • xx2 • t

(continued)

Coding Standard Template (continued)

<table>
<tr><td>Comments</td><td>Document the code so that the reader can understand its operation.
Comments should explain both the purpose and behavior of the code.
Comment variable declarations to indicate their purpose.</td></tr>
<tr><td>Good Comment</td><td><code>/*
 * computeCoefficient
 * Propósito: calcular C usando la función Gamma.
 */</code></td></tr>
<tr><td>Bad Comment</td><td><code>// exponent</code></td></tr>
<tr><td>Major Sections</td><td>Precede major program sections by a block comment that describes the processing that is done in the next section</td></tr>
<tr><td>Example</td><td><code>/* ==============================
   Atributos
   ============================== */

/* ==============================
   Métodos Públicos
   ============================== */

/* ==============================
   Métodos Privados
   ============================== */</code></td></tr>
<tr><td>Blank Spaces</td><td>Write programs with sufficient spacing so they do not appear crowded.
Separate every program construct with at least one space.</td></tr>
<tr><td>Indenting</td><td>Indent every level of brace from the previous one.
Open and closing braces should be on lines by themselves and aligned with each other.</td></tr>
<tr><td>Indenting Example</td><td><code>if (x > 0)
{
    y = 2;
}</code></td></tr>
<tr><td>Capitalization</td><td>Capitalized all defines.
Lowercase all other identifiers and reserved words.
Messages being output to the user can be mixed-case so as to make a clean user presentation.</td></tr>
</table>

Capitalization Example	`public static final double PI_VALUE = 3.141592;`
	`private double dblW;`
	`public void computeW() { ... }`